DEPLO - FROM REPO TO REALITY

A report submitted for the course named Project - II (CS3202)

Submitted By

SHANTNU BABAN KUMBHIRKAR
SEMESTER - VI
220101075

Supervised By

DR. KHOIROM MOTILAL SINGH



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF INFORMATION TECHNOLOGY SENAPATI, MANIPUR

APRIL, 2025

Declaration

In this submission, I have expressed my idea in my own words, and I have adequately cited and referenced any ideas or words that were taken from another source. I also declare that I adhere to all principles of academic honesty and integrity and that I have not misrepresented or falsified any ideas, data, facts, or sources in this submission. If any violation of the above is made, I understand that the institute may take disciplinary action. Such a violation may also engender disciplinary action from the sources which were not properly cited or permission not taken when needed.

SHANTNU BABAN KUMBHIRKAR
220101075

DATE :-

Department of Computer Science and Engineering
Indian Institute of Information Technology Senapati, Manipur

Dr. Khoirom Motilal Singh
Assistant Professor

Email: motilal@iiitmanipur.ac.in
Contact No: +91 7085242211

To Whom It May Concern

This is to certify that the project report entitled **" DEPLO - FROM REPO TO REALITY ",** submitted to the department of Computer Science and Engineering, Indian Institute of Information Technology Senapati, Manipur in partial fullfillment for the award of degree of Bachelor of Technology in Computer Science and Engineering is record bonafide work carried out by **Shantnu Baban Kumbhirkar** bearing roll number 220101075.

Signature of Supervisor

Dr. Khoirom Motilal Singh

Signature of the Examiner 1 .....

Signature of the Examiner 2 .....

Signature of the Examiner 3 .....

Signature of the Examiner 4 .....

Abstract

Deplo – From Repo To Reality is a deployment automation platform designed to simplify the transformation of source code from remote repositories into fully operational, production-grade services. The system eliminates the complexities of manual deployment by integrating a web-based interface with a backend orchestration engine grounded in modern software architecture principles. with AWS Elastic Container Service (ECS), which facilitates the automated deployment of containerized applications. Upon deployment initiation, the system triggers an ECS task that clones the target repository, builds the application, and deploys it within an isolated environment. Secure management of environment variables and customizable build configurations ensure consistency and reproducibility across deployments.Deplo is architected with an emphasis on modularity, scalability, and fault tolerance. By automating the deployment lifecycle—from repository ingestion to live application hosting—the platform reduces operational overhead, minimizes human error, and accelerates development cycles. This project demonstrates the effective application of cloud-native technologies, event-driven architecture, and infrastructure-as-code principles, offering a robust and extensible solution to contemporary software delivery challenges.

Acknowledgement

I express my profound gratitude to Dr. Khoirom Motilal Singh, a member of our esteemed faculty, for his unwavering guidance, continuous supervision, and invaluable provision of project-related information. His support played a pivotal role in the successful completion of this project. While I have invested significant effort in this endeavor, it is crucial to acknowledge the collective contribution of the faculty members within the Computer Science and Engineering Department at IIIT Manipur. Their keen interest in my project and consistent guidance were instrumental in bringing this project to fruition. I extend my heartfelt thanks to all faculty members for their encouragement, which played a crucial role in making this project a success. I also extend my deepest appreciation to my parents for providing me with all the necessary means and encouraging me in my pursuits of both education and life. Their unwavering support, whether by fulfilling my needs or nurturing my ambitions, has been invaluable in getting me this far. I would like to thank them for always believing in me and being my strongest support.

Shantnu Baban Kumbhirkar

Contents

List of Figures **4**

List of Tables **5**

1 Introduction **6**
1.1 Outline 7
1.2 Problem Statement 8
1.3 Motivation 9
1.4 Purpose 10

2 Literature Survey **11**
2.1 GitOps and Repository-Centric Deployment Automation 12
2.2 Event-Driven Build Orchestration with Apache Kafka 13
2.3 Scalable Container Deployment Using Amazon ECS and Fargate 14
2.4 Backend State Management and ORM Efficiency with Prisma . 14
2.5 Real-Time Logging and Analytics with ClickHouse 15
2.6 Developer-Centric Interface Design Using Next.js 16

3 Requirement Engineering **17**
3.1 Stakeholder Identification 18
3.1.1 Developers as Primary Users 18
3.1.2 System Administrators 19
3.1.3 Platform Backend Services 19
3.2 Functional Requirements 20
3.2.1 GitHub Repository Integration 20
3.2.2 Deployment Triggering 21
3.2.3 Real-time Build Monitoring 21
3.2.4 Project and User Management 22
3.2.5 Log Analytics and Visualization 22
3.3 Non-Functional Requirements 23
3.3.1 Performance 23
3.3.2 Scalability 23
3.3.3 Usability 23
3.3.4 Reliability 24

3.4 System Constraints . . . 24
3.4.1 Infrastructure Dependency . . . 24
3.4.2 Authentication Handling . . . 24
3.4.3 Real-time Communication . . . 25
3.4.4 Cross-platform Deployability . . . 25
3.5 Prioritization of Requirements . . . 25
3.6 Use Case Analysis . . . 26
3.6.1 Actors . . . 26
3.6.2 Use Cases . . . 27

4 System Design **30**
4.1 System Architecture . . . 31
4.1.1 User Interface (UI) and Configuration Management . . . 31
4.1.2 Backend Services and Deployment Orchestration . . . 31
4.1.3 Deployment Engine . . . 32
4.1.4 GitHub Integration . . . 32
4.1.5 Security and Authentication . . . 32
4.1.6 Component Interaction . . . 33
4.2 Data Flow Diagram (DFD) . . . 34
4.2.1 DFD Level 0: Context Diagram . . . 34
4.2.2 DFD Level 1: High-Level Functional Decomposition . . . 35
4.3 Sequence Diagram . . . 37
4.4 Flowchart . . . 40

5 Implementation **43**
5.1 Repository Integration and Authentication . . . 44
5.2 Deployment Lifecycle Management . . . 45
5.3 Containerization and Build System . . . 46
5.4 Real-Time Logging and Event Streaming . . . 48
5.5 Frontend Interface and Developer Dashboard . . . 49
5.6 Database and State Management . . . 50

6 Results **52**
6.1 Scalable Static Hosting via Dynamic Subdomain Routing . . . 53
6.2 Automation of End-to-End Deployment Workflow . . . 54
6.3 Real-Time Deployment Feedback and Preview Accessibility . . . 56

7 Conclusion **58**
7.1 Summary of Key Features . . . 58
7.2 Challenges and Limitations . . . 60
7.3 Future Enhancements . . . 60
7.4 Final Thoughts . . . 61

Bibliography **62**

List of Figures

3.1 Use Case Diagram 29

4.1 DFD Level 0 34
4.2 DFD Level 1 36
4.3 Sequence Diagram 38
4.4 Flowchart 41

6.1 Deployment Input Interface Enabling Dynamic Subdomain Hosting 54
6.2 Automation of End-to-End Deployment Workflow 56
6.3 Real-Time Deployment Feedback and Preview Accessibility . . . 57

List of Tables

3.1 Prioritization of Requirements 26
3.2 Use Case Table 28

Chapter 1

Introduction

In the evolving landscape of cloud-native application development, the demand for automated, scalable, and developer-friendly deployment solutions is rapidly increasing [1]. Deplo – From Repo to Reality is an open-source, self-hosted deployment platform designed to address this demand by providing an end-to-end pipeline for deploying static websites and frontend applications directly from Git repositories. Drawing architectural influence from modern deployment services, Deplo emphasizes modularity, extensibility, and automation, thereby streamlining the transition from code to production with minimal developer intervention.

Deplo's architecture adopts a microservices-based model composed of three core components: the API server, build server, and static asset reverse proxy. The API server coordinates deployment workflows, manages RESTful endpoints, and integrates with container orchestration platforms such as AWS ECS. The build server, operating within Docker containers, automates repository cloning, application compilation, and artifact deployment to S3-compatible object storage. The reverse proxy routes incoming requests to the appropriate hosted assets based on subdomain or custom domain mappings, ensuring scalable and performant content delivery. The front end of Deplo is built using modern web technologies, including Next.js with React Server Components and Tailwind CSS for a clean, responsive UI. User authentication is handled via NextAuth and Clerk, while real-time deployment feedback is enabled through a WebSocket-based logging system. The platform supports custom domain configuration with automated DNS resolution, and includes fine-grained security controls, tokenized API access, and rigorous input validation, all backed by a PostgreSQL database managed through Prisma ORM.

Deplo presents a production-ready solution suitable for developers, small teams, and educational settings aiming to implement or understand continuous integration and deployment workflows. By abstracting infrastructure complexity and automating orchestration, it enhances user autonomy while promoting best practices in full-stack development and modern DevOps.

1.1 Outline

Deplo – From Repo to Reality is systematically structured into multiple core components, each fulfilling a distinct and specialized function within the continuous deployment pipeline. This section presents a formal breakdown of the system architecture and its integral modules, detailing their individual responsibilities and collaborative interactions. The overall deployment flow is governed by the coordination between the API server, build infrastructure, and reverse proxy, enabling a fully automated transition from code repository to live web application.

1. **API Server and System Orchestration**
   The API server functions as the central orchestrator of the platform, responsible for handling HTTP requests, managing user interactions, and coordinating deployment workflows [2]. Developed with Node.js and Express, this component exposes RESTful endpoints for initiating builds, configuring projects, and managing custom domains. It interfaces with the backend database via Prisma ORM to maintain persistent data structures for users, deployments, and domain mappings, ensuring system-wide consistency.

2. **Automated Build Server and Deployment Engine**
   The build server encapsulates the logic for automated deployment execution. Upon receiving a trigger, it clones the specified GitHub repository, installs dependencies, builds the application, and stores the resulting static assets in an object storage service (e.g., S3-compatible). These operations are executed within isolated Docker containers, allowing for concurrency and reproducibility. The use of dynamic port allocation and containerized environments ensures scalable and conflict-free execution.

3. **Reverse Proxy for Dynamic Content Delivery**
   The reverse proxy module facilitates content delivery by dynamically routing incoming requests based on subdomain or custom domain configurations. It serves static files directly from cloud storage, ensuring low-latency access and domain-aware resolution. This component also integrates with the platform's domain management logic, enabling automatic mapping and secure access to deployed resources.

4. **Frontend Interface and User Dashboard**
   The client-facing dashboard, implemented using Next.js 13 and React Server Components, offers developers an intuitive and responsive interface for managing deployments [3]. Tailwind CSS ensures design consistency, while authentication and session handling are managed

via Clerk and NextAuth. Through this interface, users can initiate deployments, view logs, assign domains, and track project states, all in real time.

5. **Real-Time Logging and Feedback Mechanism**
   To enhance observability, a WebSocket-based logging infrastructure delivers real-time deployment logs from the build server to the frontend [4]. This mechanism enables users to monitor deployment progress, identify build failures, and access detailed logs for debugging. The system ensures transparency and traceability throughout the deployment lifecycle.

6. **Database and Persistent State Management**
   Persistent data storage is handled by PostgreSQL, interfaced via Prisma ORM [5]. The database maintains structured records of user accounts, project metadata, deployment logs, and domain configurations. This persistent layer ensures data reliability, supports historical queries, and provides a foundation for further analytics and reporting features.

This modular architecture ensures Deplo's scalability, maintainability, and extensibility. Each component operates independently while contributing to a cohesive and robust deployment framework. The system is designed to support modern development workflows, offering developers a high degree of control and automation within a self-hosted environment.

1.2 Problem Statement

In contemporary web development, the growing reliance on continuous integration and continuous deployment (CI/CD) necessitates deployment solutions that are automated, scalable, and developer-oriented. Despite the increasing maturity of CI/CD methodologies, developers—particularly those working within self-hosted, academic, or resource-constrained environments continue to face significant challenges in implementing comprehensive deployment pipelines. These challenges include managing containerized builds, automating artifact storage, configuring domain routing, and enabling real-time deployment monitoring.

Existing solutions, while functional, often lack transparency, extensibility, or the ability to be fully self-hosted. As a result, they limit developers' control over deployment workflows and inhibit deeper understanding of the underlying infrastructure. Additionally, the fragmentation of tooling in the deployment ecosystem leads to increased system complexity, inconsistent integration, and a higher barrier to entry for those seeking to implement or customize deployment pipelines.

The core issue addressed by this project is the absence of a unified, open-source framework that offers a complete deployment lifecycle—from repository integration and containerized builds to static asset distribution and domain management—within a modular and self-hosted architecture. There is a critical need for a solution that consolidates these components into a coherent system, enabling full-stack deployment without sacrificing flexibility or operational transparency.

Deplo – From Repo to Reality responds to this need by providing a comprehensive deployment platform designed to support modern web application delivery. It emphasizes modular design, operational clarity, and extensibility, thereby offering a robust foundation for developers seeking autonomy in managing their deployment infrastructure. The project aims to bridge the gap between source code and production environments, facilitating a more accessible and customizable approach to CI/CD in a self-managed context.

1.3 Motivation

The development of Deplo – From Repo to Reality is driven by the need for a flexible, cost-effective deployment solution that offers comprehensive control over the deployment process. While numerous commercial deployment platforms exist, they often impose limitations in terms of customization, transparency, and self-hosting capabilities. These constraints hinder developers who require greater flexibility and autonomy over their deployment workflows, particularly in academic, small-team, or resource-constrained environments. Deplo addresses these limitations by providing a fully customizable platform that allows developers to manage and deploy applications independently, without relying on external providers. The system's modular architecture is designed to be adaptable, enabling users to tailor the deployment pipeline to meet their specific needs. This makes Deplo an ideal solution for developers seeking to maintain control over their infrastructure while minimizing the costs associated with proprietary deployment platforms. In addition, Deplo aims to simplify and automate the deployment process, bridging the gap between development and operations. By streamlining tasks such as repository integration, build automation, and static asset deployment, Deplo reduces the manual overhead typically involved in deployment workflows. This automation allows developers to focus on core application development, while the platform handles the complexities of the deployment lifecycle.

Furthermore, Deplo serves an educational purpose, providing an accessible tool for students and developers to learn about continuous integration, containerization, and cloud-based application delivery. By offering a transparent and hands-on approach to deployment, Deplo enhances understanding of modern DevOps practices and infrastructure management, empowering users to build, deploy, and scale applications efficiently.

1.4 Purpose

The primary purpose of the Deplo – From Repo to Reality project is to design and implement a comprehensive, automated, and scalable deployment platform that facilitates the seamless transformation of source code repositories into live, production-ready web applications. This initiative seeks to streamline the deployment pipeline by integrating code compilation, containerized build environments, and static asset delivery within a unified system.

A central objective of the project is to minimize manual intervention in the deployment process, thereby enhancing efficiency and developer productivity. This is achieved through a modular microservices architecture encompassing build automation, container orchestration using Docker, and static content hosting via S3-compatible object storage [2]. The inclusion of a custom reverse proxy ensures reliable routing of requests based on user-defined subdomains or domains.

The system is engineered for scalability and maintainability, supporting a wide range of deployment scenarios from individual projects to enterprise-level applications. Each deployment is executed within isolated containers to guarantee environmental consistency, fault tolerance, and reproducibility across varied infrastructures.

An additional aim is to embody and promote modern software engineering practices. By leveraging contemporary technologies—such as AWS ECS, Docker, Next.js, and Prisma—the project demonstrates a practical application of best practices in automated deployment, infrastructure abstraction, and service design [1]. This enables developers to focus on application development while the platform transparently manages operational concerns.

The project also emphasizes usability through a streamlined frontend interface that facilitates repository integration, build monitoring, and deployment management. This intuitive design lowers the barrier to entry for users unfamiliar with complex DevOps workflows, thereby broadening accessibility.

Moreover, Deplo aspires to serve as an instructional resource for software engineering students and professionals. Through its clean, well-structured codebase and adoption of industry-standard tools and frameworks, the project exemplifies principles of modular design, abstraction, and system integration.

In summary, Deplo – From Repo to Reality aims to redefine the deployment process for modern web applications by delivering an automated, user-friendly, and scalable solution. It contributes to both practical development workflows and academic learning, positioning itself as a model for infrastructure-centric software engineering.

Chapter 2

Literature Survey

Deplo – From Repo To Reality is a full-stack, automated deployment platform designed to streamline the process of building, deploying, and managing web applications directly from source code repositories. The system establishes a robust continuous deployment pipeline that integrates version control, build orchestration, containerization, and real-time monitoring within a unified architecture [2]. By leveraging a modular and scalable design, it offers a comprehensive solution for developers seeking to automate infrastructure provisioning and deployment workflows [6].

At the core of the platform is an event-driven architecture facilitated by Apache Kafka, which decouples services and enables asynchronous execution of deployment tasks. Upon detection of changes in a connected source code repository, the platform initiates build jobs, which are queued and dispatched to isolated environments. These environments are provisioned dynamically using Amazon ECS Fargate, enabling scalable, containerized executions without requiring manual infrastructure management.

The backend infrastructure is built with Express.js and utilizes Prisma ORM to interface with the underlying database in a type-safe and schema-driven manner. This layer is responsible for managing user authentication, project metadata, build logs, and deployment records. Prisma ensures consistency, maintainability, and efficient querying while supporting complex relational data structures relevant to the deployment lifecycle.

To support real-time analytics and observability, the system integrates ClickHouse, a high-performance column-oriented database optimized for time-series data and log aggregation [7]. This enables efficient storage and retrieval of build and deployment logs, along with fine-grained telemetry visualization and system health monitoring. The platform thereby achieves comprehensive operational insight, enhancing its fault detection and debugging capabilities.

The user interface, developed with the Next.js framework, delivers a responsive, server-rendered dashboard tailored for developers. It enables users

to manage repositories, monitor build pipelines, and inspect deployment status in real time. The interface emphasizes performance, clarity, and interactivity, incorporating authentication, authorization, and API-driven state synchronization for a seamless user experience.

This system adheres to the principles of GitOps, where infrastructure and deployment logic are declaratively defined and version-controlled. By automating traditionally manual steps such as containerization, configuration, and cloud orchestration, Deplo significantly reduces operational overhead and promotes consistency across environments.

Historically, software deployment processes evolved from monolithic scripting approaches to modular, cloud-native solutions emphasizing scalability and reproducibility. The advent of containerization, coupled with managed orchestration platforms and event-driven systems, has transformed deployment pipelines into reliable, automated services. Similarly, real-time observability and analytics—once considered advanced enterprise features—are now fundamental components of modern DevOps platforms, driving proactive system management and user feedback loops.

Deplo – From Repo To Reality synthesizes these developments into a cohesive platform that exemplifies the convergence of contemporary deployment strategies. Through its integration of asynchronous build orchestration, containerized infrastructure, real-time analytics, and developer-oriented design, the project provides a scalable and efficient pathway from source code to production deployment. It reflects a pragmatic and forward-looking approach to DevOps automation, contributing to the ongoing evolution of software delivery systems in cloud-native environments.

2.1 GitOps and Repository-Centric Deployment Automation

Deplo – From Repo To Reality adopts a GitOps-based approach to streamline deployment automation by using Git repositories as the central source of truth for both application code and infrastructure configuration. This methodology enables declarative, version-controlled, and reproducible workflows, minimizing manual intervention and promoting operational consistency [8].

The platform leverages webhook integrations with Git-based version control systems to monitor repository events such as pushes and commits. These events automatically trigger the deployment pipeline, which retrieves project metadata stored in a relational database through a type-safe Prisma ORM layer. This metadata includes repository details and associated user configurations.

Upon detecting a repository event, the backend service—built on Express.js initiates the build process by dispatching tasks to an Apache Kafka-based

event stream. This asynchronous design decouples the build execution from the triggering event, enabling scalability and system resilience. The build engine clones the repository, parses a configuration file (e.g., deplo.json), and executes the defined build steps within containerized environments on Amazon ECS Fargate. These ephemeral containers ensure reproducibility and environment isolation, key principles of GitOps.

By automating deployments through declarative configurations and continuous synchronization with version control, the system achieves rapid, reliable, and auditable software delivery. This integration of GitOps not only accelerates development cycles but also enhances traceability, rollback capability, and infrastructure transparency—advancing modern DevOps practices within a repository-centric paradigm.

2.2 Event-Driven Build Orchestration with Apache Kafka

Deplo – From Repo To Reality utilizes an event-driven architecture centered around Apache Kafka to manage and scale its build orchestration process. Kafka serves as a high-throughput message broker, decoupling components and enabling asynchronous task execution across distributed services [9].

When a repository event is detected, a build request is serialized and published to a Kafka topic. This request encapsulates metadata such as the project ID, repository URL, and target branch. Dedicated Kafka consumers, implemented as worker services, subscribe to this topic, retrieve the message, and initiate the build by cloning the repository and interpreting the project's declarative configuration file (deplo.json).

This architecture enables concurrent and isolated build executions, supporting horizontal scalability through additional consumers. Kafka's durability and message replay capabilities ensure reliable task processing and fault recovery, preserving build integrity even under transient failures.

Moreover, Kafka provides a centralized stream of build-related events, facilitating extensibility for logging, analytics, and monitoring without disrupting core functionalities. This modularity aligns with cloud-native design principles, enhancing maintainability and future system adaptability.

In essence, Kafka enables robust, scalable, and resilient orchestration within Deplo, ensuring consistent build execution in a decoupled and event-driven deployment ecosystem.

2.3 Scalable Container Deployment Using Amazon ECS and Fargate

Deplo – From Repo To Reality employs Amazon ECS (Elastic Container Service) in combination with AWS Fargate to provide a scalable and serverless solution for containerized deployments. This architecture enables the dynamic execution of build and deployment tasks within isolated containers, optimizing resource utilization and scalability.

ECS orchestrates container management across clusters, while Fargate abstracts the underlying infrastructure, eliminating the need for manual server management. Containers are provisioned and executed in isolated environments, ensuring consistency and security across different deployment stages [10].

Upon the initiation of a build request, Fargate automatically provisions a task for each deployment, defining parameters such as CPU, memory, and networking configuration based on application requirements. This ensures optimal resource allocation for each containerized build. The system's scalability is enhanced by Fargate's serverless nature, which adjusts resource allocation automatically to meet fluctuating demands. This approach efficiently handles variable workloads, ensuring cost-effectiveness and responsive scaling without manual intervention.Furthermore, ECS provides built-in resilience through automatic container re-launching in the event of failure, maintaining high availability and operational continuity. Health checks and auto-scaling policies further enhance the reliability and fault tolerance of the deployment process.

By integrating ECS and Fargate, Deplo – From Repo To Reality delivers a robust, flexible, and scalable deployment solution that aligns with modern cloud-native practices, ensuring efficient and reliable application delivery.

2.4 Backend State Management and ORM Efficiency with Prisma

Deplo – From Repo To Reality utilizes Prisma as its Object-Relational Mapping (ORM) solution to efficiently manage backend state and facilitate database interactions. Prisma's type-safe query engine and schema management tools streamline data handling, ensuring consistency and reliability in backend operations [11].

Prisma enables the declarative definition of data models through its schema, which is closely integrated with the application's logic. This ensures that relational data, such as user profiles, repository details, and deployment configurations, is accurately represented and queried. The schema is automatically synchronized with the database through Prisma's migration system, simplifying database management and reducing the risk of inconsistencies during

schema changes.

The platform's state management relies on Prisma's efficient querying capabilities to handle complex relationships between entities. For example, user data and associated repositories are managed through type-safe queries, ensuring data integrity and reducing the potential for errors in state transitions.

Prisma's migration system further enhances the application by ensuring smooth database versioning and the safe application of schema updates across environments. This eliminates the need for manual interventions and ensures that the system remains consistent, even as the underlying data model evolves.

Moreover, Prisma's optimized query engine reduces database query overhead, improving the application's performance. Its type-safety guarantees at compile time help identify and resolve potential issues related to data handling before runtime, significantly improving system reliability and reducing errors.

By abstracting the database layer and offering a clear and efficient API for data interaction, Prisma also promotes extensibility and modularity. It allows seamless integration with additional services and future system expansions, ensuring that Deplo remains flexible and scalable.

2.5 Real-Time Logging and Analytics with ClickHouse

Deplo – From Repo To Reality integrates ClickHouse, a high-performance columnar database, to facilitate real-time logging and analytics within the deployment pipeline [12]. This solution enables efficient aggregation, querying, and analysis of large volumes of log data generated during build and deployment processes [13].

ClickHouse's columnar storage model optimizes the retrieval of large datasets, enabling fast queries for performance monitoring, error tracking, and operational insights. This architecture allows for rapid aggregation of log events across multiple dimensions, such as deployment status, repository, and time, thus providing valuable real-time insights into system health and performance. Deployment events, build outcomes, error logs, and other relevant operational data are ingested into ClickHouse and stored in a compressed columnar format. This structure supports quick data retrieval and high-performance queries, enabling users to monitor deployment progress, track build success rates, and swiftly identify failures or bottlenecks in the pipeline.

Real-time logging and analytics enhance system observability, providing operators with the necessary tools to quickly detect and resolve issues. ClickHouse's speed and scalability support rapid querying, empowering stakeholders to make informed decisions and take corrective actions promptly, ensuring the reliability of the deployment process.

Additionally, ClickHouse supports advanced querying capabilities such as filtering, grouping, and aggregation. These features allow for customized reporting and the creation of dashboards that monitor key performance indicators (KPIs), offering stakeholders a comprehensive view of deployment metrics.

2.6 Developer-Centric Interface Design Using Next.js

Deplo – From Repo To Reality utilizes Next.js as the core framework for frontend development, enabling the creation of an efficient, developer-centric user interface [14]. As a React-based framework, Next.js supports server-side rendering (SSR), static site generation (SSG), and API routes, facilitating an optimized user experience for developers interacting with the platform.

The platform's interface is designed with a focus on simplicity, usability, and performance. It provides developers with an organized and intuitive view of the deployment pipeline, from repository integration to deployment execution. The use of Next.js ensures fast rendering and real-time updates, allowing developers to monitor and manage deployments with minimal latency.

Next.js's file-based routing system and automatic code splitting ensure efficient performance and seamless navigation. The framework's API routes simplify backend interaction, reducing the complexity often associated with frontend-backend integration. This modular approach streamlines development and minimizes overhead.

Next.js also enhances the platform with server-side rendering, which improves the dynamic content delivery and optimizes load times, thus contributing to a responsive and fast user experience. These features are particularly important for developers who need quick access to deployment status, logs, and other operational data without delays.

Moreover, dynamic content updates are seamlessly integrated, allowing real-time feedback as developers initiate builds or deployments. This ensures that users are consistently informed about the status of their operations, eliminating the need for manual refreshing or navigation.

By leveraging React alongside Next.js, the interface is both highly modular and maintainable, supporting future scalability and the easy addition of new features. The developer-centric design further streamlines deployment management, log monitoring, and overall system interaction, enhancing developer productivity.

Chapter 3

Requirement Engineering

Requirements engineering plays a pivotal role in the development of dependable and scalable software systems by ensuring that system objectives are clearly defined, systematically analyzed, and accurately implemented [7]. For Deplo – From Repo To Reality, the requirements engineering process establishes the foundational structure of a platform that automates the transition from a source code repository to a fully deployed, production-ready application.

The platform adopts a modular, service-oriented architecture, emphasizing automation, scalability, and usability. The frontend is implemented using Next.js and React, offering an intuitive interface through which users can submit repository URLs, trigger deployments, and monitor progress. The user interface is minimalistic yet functional, guiding users seamlessly through the deployment process while providing real-time feedback on build and deployment states.

The backend, developed with Express.js, serves as the orchestration layer responsible for managing core operations. It validates repository inputs, triggers deployment workflows, and communicates with underlying infrastructure services. It leverages Kafka to facilitate asynchronous, event-driven communication, thereby enabling concurrent deployment pipelines and resilient state management [9]. RESTful APIs exposed by the backend synchronize system states between the user interface and backend processes, ensuring real-time consistency and responsiveness.

Persistent data storage is handled via Prisma, which provides a type-safe and schema-driven abstraction over the database layer. Analytical capabilities are integrated through ClickHouse, which supports high-performance logging, metric collection, and deployment analytics. This integration enables enhanced observability and performance optimization over time.

The architecture emphasizes decoupling, modularity, and scalability. The use of stateless services, containerized execution environments, and event-based processing supports horizontal scalability and fault tolerance. Each

module is designed with clear separation of concerns, facilitating maintainability, testing, and future extensibility.

The requirements engineering methodology applied to Deplo – From Repo To Reality ensures a well-defined mapping between system behaviors, architectural components, and user interactions. It captures the complexities of modern deployment workflows and codifies them into a cohesive, automated system that minimizes manual intervention and optimizes development-to-deployment efficiency. Through rigorous specification and architectural discipline, the platform delivers a robust and extensible solution for modern software deployment needs.

3.1 Stakeholder Identification

Stakeholder identification is a critical component in the requirements engineering process of Deplo – From Repo To Reality, ensuring that the system's architecture, features, and workflows align with the roles and expectations of all interacting entities. The platform supports a range of stakeholders, each fulfilling distinct responsibilities throughout the deployment lifecycle. These include primary users initiating deployments, infrastructure managers overseeing system reliability, and automated backend services enabling seamless orchestration.

3.1.1 Developers as Primary Users

Stakeholder Role

Developers represent the principal user group of the platform. Their interaction is facilitated through a web-based interface developed with Next.js, which enables them to submit repository URLs, initiate deployments, and monitor build and deployment progress. The platform abstracts the complexities of infrastructure management, allowing developers to deploy applications with minimal configuration.

System Interaction

Upon user input, the frontend dispatches deployment requests via API endpoints defined in express/routes/deploy.js. These requests trigger backend services responsible for validating the repository, initiating ECS task execution, and reporting real-time deployment status. The user interface dynamically reflects system feedback, ensuring transparency and responsiveness.

Responsibilities

- Initiating deployments by submitting GitHub repository URLs.
- Monitoring real-time build logs and deployment status.
- Reviewing historical deployments through persistent dashboards.
- Responding to system feedback in cases of deployment success or failure.

3.1.2 System Administrators

Stakeholder Role

System administrators oversee the underlying deployment infrastructure. Their responsibilities include configuring and maintaining AWS ECS clusters, managing task definitions, and ensuring system availability and performance. Although they do not interact directly with the user interface, their role is crucial to maintaining a reliable and secure deployment environment.

System Interaction

Administrative functions are primarily exercised outside the application codebase but are reflected in backend logic such as express/controllers/deployController.js and utils/awsUtils.js, where ECS task definitions and execution are programmatically triggered. Administrators ensure these configurations remain valid, optimized, and secure.

Responsibilities

- Maintaining ECS cluster configurations and task definitions.
- Managing authentication, secrets, and permissions via IAM.
- Monitoring infrastructure health and scaling behaviors.
- Ensuring continuous availability and fault tolerance of deployment services.

3.1.3 Platform Backend Services

Stakeholder Role

Platform backend services function as autonomous agents within the system, facilitating internal communication and task orchestration. These services

include Kafka-based message producers and consumers, background workers, and webhook handlers. They operate asynchronously to ensure modularity, scalability, and fault isolation across the deployment pipeline.

System Interaction

Kafka is employed as the core messaging infrastructure, as demonstrated in express/kafka/producer.js and consumer.js. Deployment events are broadcasted through Kafka topics and consumed by subscribed services to execute subsequent tasks—such as updating UI state, persisting logs, or triggering notifications.

Responsibilities

- Automating deployment workflows through event-driven communication.
- Coordinating internal services via message queues and asynchronous processing.
- Maintaining consistency between deployment states and system outputs.
- Ensuring reliability and scalability of background task execution.

3.2 Functional Requirements

3.2.1 GitHub Repository Integration

The integration of GitHub repositories is a fundamental requirement for Deplo – From Repo To Reality, enabling the platform to automatically trigger deployments based on code changes. The system must satisfy the following requirements:

- **Webhook Listening:** The platform integrates with GitHub repositories to automate the deployment process. It listens to push events emitted by GitHub through webhooks, enabling the system to respond dynamically whenever code changes are committed to a repository. This integration ensures that the deployment pipeline is triggered automatically and aligns with the latest version of the codebase.
- **Repository Metadata Extraction:** Upon receiving a webhook notification, the system extracts relevant metadata from the repository, including repository details (e.g., name, owner) and build configurations (e.g., build scripts, environment variables). This data is used to configure the deployment pipeline and ensure compatibility between the

application and the target environment. The extraction process, implemented within the backend code in express/routes/deploy.js, is crucial for ensuring that the deployment workflow is aligned with the repository's structure and configuration.

3.2.2 Deployment Triggering

Automated deployment triggering is central to the functionality of Deplo – From Repo To Reality. The system must ensure the seamless initiation of deployment workflows in response to GitHub push events. The following requirements must be met:

- **Programmatic Task Invocation:** Following the receipt of a webhook event, the platform triggers the deployment process by programmatically invoking an AWS ECS task. This is achieved via backend code in express/controllers/deployController.js, where the system dynamically provisions and manages containers based on the extracted repository metadata and build configurations. The ECS task execution is initiated with the appropriate configurations, ensuring that the deployment pipeline begins promptly and accurately.
- **Dynamic Container Configuration and Runtime Management:** The deployment process involves dynamically configuring and managing containerized environments. Based on the metadata extracted from the repository, the platform automatically adjusts container settings, such as resource allocation (CPU, memory) and environment variables, to match the needs of the repository. The ECS cluster and task definitions are dynamically created and configured in real time, ensuring that each deployment is tailored to the application's requirements.

3.2.3 Real-time Build Monitoring

Real-time monitoring of builds and deployments is a critical requirement to ensure developers and system administrators are kept informed of progress and potential issues. The following functionalities must be implemented:

- **Streaming Logs to Frontend:** Real-time build monitoring is a critical feature of Deplo – From Repo To Reality. The platform utilizes Kafka for streaming logs from the backend to the frontend. This enables continuous log updates to be pushed to the user interface, allowing developers to monitor the status of the build and deployment in real time. Alternatively, a polling mechanism is implemented as a fallback to ensure that users are always aware of the deployment status, even in the absence of a Kafka connection.

- **Visualizing Build and Deployment Progress:** The frontend is designed to visualize the deployment process in a clear and intuitive manner. Using dynamic components, the frontend displays real-time updates on build progress, including details on the current deployment stage, logs, and any errors or warnings encountered during the process. This functionality, which is handled via API calls between the backend and frontend, ensures transparency and provides users with the necessary information to track the deployment lifecycle.

3.2.4 Project and User Management

Effective management of deployment projects and associated user data is essential for providing a personalized and organized experience. The following requirements are key to ensuring that projects and users are properly managed:

- **Managing Deployment Projects:** The platform allows users to create and manage deployment projects. Each project corresponds to a GitHub repository and is associated with specific deployment configurations, including container settings, build commands, and environment variables. Users can view the status of ongoing deployments, trigger new deployments, and review past deployment histories associated with their projects.
- **Associating User Data with Deployment History:** User management is integrated with deployment tracking, allowing each user to view their associated deployment history. This includes a record of previously initiated deployments, along with the outcomes (success or failure) and relevant logs. This data is stored in a structured format, enabling users to track their progress over time and revisit past projects. The persistence and retrieval of this information are handled through backend services utilizing Prisma for data management and ClickHouse for log analytics.

3.2.5 Log Analytics and Visualization

The ability to analyze and visualize deployment logs is crucial for monitoring deployment performance and identifying potential issues. The system must support the following requirements:

- **Persisting Logs in ClickHouse:** The platform incorporates ClickHouse for high-performance storage and analysis of deployment logs. Each deployment's logs are captured and persisted in a ClickHouse database, ensuring that data is efficiently stored and indexed for fast retrieval and analysis. This integration allows the system to handle

large volumes of log data without compromising on performance, which is crucial for real-time deployment monitoring and post-deployment analytics.

- **Displaying Analytical Data in the Frontend Dashboard:** The logs stored in ClickHouse are subsequently processed and visualized on the frontend dashboard. This enables users to gain insights into deployment performance, error rates, and build durations. Through data aggregation and visualization, users are provided with actionable analytics that can inform optimization efforts, allowing for continuous improvement of the deployment process. The dashboard is designed to present this information in a user-friendly manner, ensuring that even complex datasets are easily interpretable.

3.3 Non-Functional Requirements

3.3.1 Performance

The system shall maintain high performance standards by ensuring rapid response to webhook triggers initiated by connected repositories. Upon receiving a deployment request, it shall promptly validate the repository structure, fetch the required metadata, and initiate the build process with minimal latency. Efficient orchestration of container lifecycles through AWS ECS shall be ensured to reduce cold-start times [15]. Furthermore, backend processes, including repository parsing, Docker image creation, and deployment pipeline execution, shall be optimized to achieve minimal end-to-end deployment durations.

3.3.2 Scalability

The architecture shall be inherently scalable, enabling concurrent handling of multiple deployment requests without service degradation. The system shall employ Kafka as a high-throughput messaging queue to decouple critical components, facilitating asynchronous processing of log streams and event-based task execution [16]. Stateless API services, containerized deployments, and horizontally scalable workers shall ensure that increasing workload volumes, such as simultaneous webhook invocations or multiple build triggers, do not compromise system stability or response times.

3.3.3 Usability

The system shall present a user-centric interface designed for intuitive interaction and minimal configuration overhead. Developers shall be able to manage deployments, monitor build progress, and receive real-time updates

through a well-structured and responsive frontend. Clear and descriptive error messages shall assist users in diagnosing issues, while live console outputs shall offer continuous feedback throughout the build and deployment lifecycle. User actions such as project linking, environment configuration, and deployment initiation shall be facilitated through a logically grouped and accessible dashboard interface.

3.3.4 Reliability

The system shall exhibit robust reliability by incorporating fault-tolerant mechanisms across all critical subsystems. Automatic task retries shall be implemented for transient failures during build or deployment phases. Persistent log storage shall ensure traceability of all execution steps, supporting post-failure diagnosis and auditability. Additionally, failure alerts shall be dispatched via integrated notification channels, enabling rapid issue resolution. The deployment lifecycle shall be resilient to partial failures, with state restoration and rollback mechanisms in place to safeguard against inconsistent deployment outcomes.

3.4 System Constraints

3.4.1 Infrastructure Dependency

The system architecture is inherently dependent on third-party infrastructure components, most notably AWS Elastic Container Service (ECS), Apache Kafka, and ClickHouse. The orchestration of containerized deployments is tightly coupled with ECS capabilities, necessitating the availability and correct configuration of related AWS services. Kafka serves as the backbone for inter-service communication and log stream queuing, while ClickHouse is employed for structured logging and analytical querying. Any degradation or unavailability in these external services may adversely affect core functionalities, including deployment execution, log delivery, and status tracking.

3.4.2 Authentication Handling

Secure handling of authentication credentials represents a critical constraint in the system. Environment variables, access tokens, and webhook secrets must be stored, retrieved, and utilized in a manner that prevents unauthorized access or leakage. Webhook sources are required to be thoroughly validated to ensure that only legitimate deployment triggers are accepted. Furthermore, any access to sensitive deployment operations must be gated behind robust authentication mechanisms to safeguard against spoofing, replay attacks, or unauthorized repository integrations.

3.4.3 Real-time Communication

The system mandates real-time communication between backend services and frontend clients for the purpose of continuous log streaming and deployment status updates. This necessitates the implementation of low-latency, event-driven communication mechanisms—such as WebSockets—that do not overwhelm network bandwidth or client-side rendering capacity. Care must be taken to throttle or batch logs intelligently to avoid excessive payload sizes, especially during verbose build processes, while maintaining a responsive and informative user experience.

3.4.4 Cross-platform Deployability

To accommodate diverse project environments, the system is required to support deployments across a wide array of technology stacks, including but not limited to Node.js, Python, and static sites. This is achieved through Docker-based isolation and execution. However, this imposes constraints on the build environments, as each project must conform to the containerized model of execution. Custom Dockerfiles, specific runtime dependencies, or language-specific configurations may introduce variability and complexity in build behavior, necessitating adaptable and modular pipeline design.

3.5 Prioritization of Requirements

In the development of "Deplo – From Repo To Reality", a systematic prioritization of requirements is imperative to ensure focused delivery of essential functionalities and progressive enhancement of the system. This section categorizes the identified requirements into three priority levels—High, Medium, and Low—based on their criticality to the core architecture, user experience, and future scalability. Each item has been meticulously examined in the context of the actual implementation artifacts and system behavior defined across the backend services, frontend interface, and deployment pipeline [17].

The above prioritization framework provides a strategic roadmap for progressive development. Emphasis is placed on establishing the robustness of the deployment workflow before extending support for advanced features. By delivering high-priority requirements first, the system ensures reliability, operability, and core functional integrity, thereby laying a solid foundation for iterative enhancements in future development stages [18].

Requirement	Priority	Justification
Repository Integration	High	Central to the system's function, enabling users to link source code repositories for deployment.
Deployment Triggering	High	A fundamental mechanism that initiates the deployment workflow via webhooks or manual actions.
ECS Task Management	High	Directly governs containerized build and deployment execution, making it critical to core operation.
Log Analytics	Medium	Enhances visibility into deployment processes but is secondary to initial deployment execution.
User Dashboard Features	Medium	Facilitates management and monitoring of deployments; essential for usability but not foundational.
Advanced Analytics	Low	Offers deeper insights into usage trends and performance, reserved for future development cycles.
Support for Custom Domains or SSL	Low	Adds value for production-grade hosting but not essential to the deployment core functionality.

Table 3.1: Prioritization of Requirements

3.6 Use Case Analysis

This Use Case Analysis delineates the functional architecture of the "Deplo – From Repo To Reality" project, by examining key interactions between system actors and components. Each use case captures specific functionalities that are derived from the system's backend and frontend implementations, including repository webhook handling, ECS-based deployment orchestration, logging infrastructure, and administrative controls. This structured analysis ensures alignment between development objectives and user expectations [19].

3.6.1 Actors

- **Developer:** The primary user responsible for initiating deployments and observing outcomes via the user interface.
- **System:** The backend services, task orchestration engine, and logging subsystems that execute core functionalities.
- **Admin:** A privileged actor tasked with monitoring deployment health, managing failures, and overseeing task operations.

3.6.2 Use Cases

Use Case 1: Push to GitHub and Trigger Deployment

- **Actor:** Developer
- **Preconditions:** The repository is linked and the webhook is configured.
- **Postconditions:** A new deployment is initiated via the backend.
- **Flow of Events:**
  1. The developer pushes new code to the linked GitHub repository.
  2. The system receives the webhook payload through the API endpoint.
  3. Metadata such as commit details and repository name are extracted.
  4. The system queues a deployment task via AWS ECS.
  5. The deployment process is initiated.

Use Case 2: View Live Deployment Logs on Dashboard

- **Actor:** Developer
- **Preconditions:** An active or completed deployment task exists.
- **Postconditions:** Deployment logs are streamed or displayed on the dashboard interface.
- **Flow of Events:**
  1. The developer navigates to the deployment dashboard.
  2. The system queries deployment status and logs from the backend.
  3. Logs are either streamed in real-time via WebSockets or fetched periodically.
  4. The dashboard displays real-time build logs, container output, and task progress.

Use Case 3: Record Deployment Logs and Metadata

- **Actor:** System
- **Preconditions:** A deployment task has been triggered and is in progress.
- **Postconditions:** Deployment data is persisted for future analytics.
- **Flow of Events:**

1. The system captures logs and container-level metadata during the ECS task execution.
2. Data such as timestamps, status, and output logs are structured and stored in the database.
3. Metadata is indexed for querying and analytical purposes.
4. This data supports future enhancements in observability and auditability.

Use Case 4: Monitor Failures and Retry Deployment

- **Actor:** Admin
- **Preconditions:** At least one deployment has failed or returned a non-zero exit status.
- **Postconditions:** The failed task is either requeued or marked with an administrative decision.
- **Flow of Events:**
  1. The admin reviews the dashboard or backend logs to identify failed tasks.
  2. The system provides relevant failure metadata and error traces.
  3. The admin triggers a retry or marks the deployment for further inspection.
  4. Upon retry, a new ECS task is initiated with the same parameters.

Use Case	Description	Actors	Preconditions	Postconditions
Push to GitHub and Trigger Deployment	Upon pushing code to a linked GitHub repository, the system listens to webhook events and triggers an automated deployment by queuing tasks through the backend infrastructure.	Developer, System	Repository is connected and webhook is configured.	Deployment is initiated and tracked.
View Live Deployment Logs	The developer can observe live or historical deployment logs through the dashboard, with real-time updates provided via WebSockets or API-based polling.	Developer, System	A deployment task is in progress or completed.	Logs are streamed or displayed on the dashboard.
Record Logs and Metadata	During deployment execution, the system captures and stores relevant logs and metadata, supporting observability, future diagnostics, and usage analytics.	System	A deployment task is running.	Logs and metadata are saved for analytics.
Monitor and Retry Failed Deployments	The admin is able to monitor failed tasks, examine failure reports, and initiate retries to ensure deployment continuity and fault recovery.	Admin, System	One or more deployments have failed.	Task is retried or flagged for manual intervention.

Table 3.2: Use Case Table

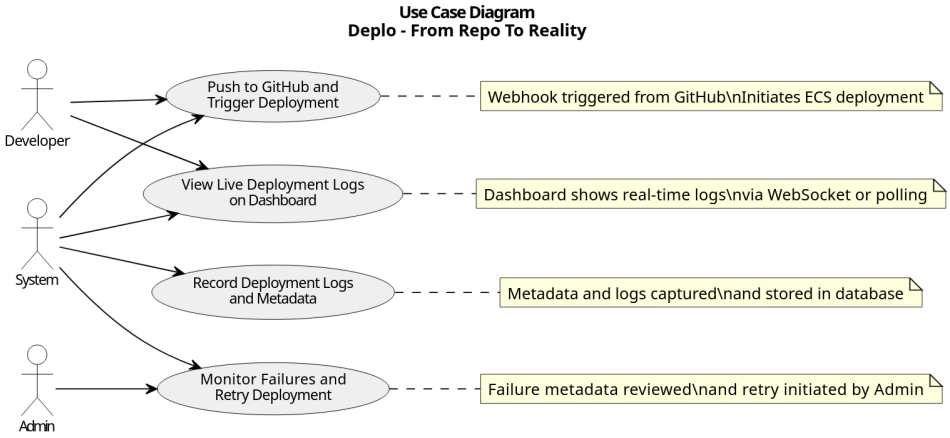


Figure 3.1: Use Case Diagram

This Use Case Analysis provides a structured foundation for the functional requirements of “Deplo – From Repo To Reality.” Each scenario described herein reflects mission-critical interactions and is tightly aligned with the actual architectural elements implemented within the codebase. Through this analytical framework, development can proceed with clarity, traceability, and strategic alignment to both user-driven needs and system resilience.

Chapter 4

System Design

Deplo - From Repo To Reality is a highly customizable and cost-effective deployment solution aimed at providing developers with enhanced control over the deployment process. The system was created with the primary goal of optimizing and streamlining the deployment pipeline, offering users the flexibility to manage the transition of their code from a repository to a live production environment. By integrating version control systems with an automated deployment pipeline, Deplo ensures efficient, hands-off deployments, minimizing the need for manual intervention. The project seeks to simplify the complexity of modern deployment workflows, which often involve coordination across multiple services and platforms.

At its core, Deplo facilitates the automatic deployment of applications directly from Git repositories. Users can link their GitHub repositories to the deployment pipeline, which triggers automatic deployments when changes are pushed to specific branches or when new pull requests are created. This functionality ensures that updates made within the repository are automatically reflected in the production environment, streamlining the development process.

The system consists of several key components that work in unison to deliver seamless deployment. The user interface (UI) allows users to configure and monitor deployment tasks with ease. Backend services manage interactions between the GitHub API and the deployment engine, which is responsible for fetching the latest repository changes, compiling the application, and deploying it to the specified production environment be it a cloud hosting platform or a private server. This architecture ensures a smooth, automated deployment experience with minimal manual oversight.

4.1 System Architecture

The architecture of Deplo - From Repo To Reality is designed to deliver an efficient, scalable, and highly flexible deployment solution. It seamlessly integrates version control systems with an automated deployment pipeline, ensuring the smooth transition of code from repositories to live production environments. This section outlines the key components, their respective roles, and their interactions within the system.

4.1.1 User Interface (UI) and Configuration Management

The user interface (UI) serves as the primary point of interaction for users, enabling them to configure and manage deployment pipelines. It is designed to be intuitive and accessible, facilitating the following key functions:

- **Repository Integration:** Users authenticate via OAuth with GitHub, linking their repositories to the deployment pipeline. This ensures secure access without exposing sensitive credentials.
- **Deployment Configuration:** Through the UI, users can specify deployment parameters, such as target environments (e.g., cloud platforms or private servers), trigger conditions (e.g., branch pushes or pull requests), and build scripts.
- **Real-time Monitoring:** The UI provides live feedback on the deployment status, including success, failure, and logs, which allows users to track the progress of their deployments in real time.

4.1.2 Backend Services and Deployment Orchestration

The backend is responsible for managing the interactions between the user interface, GitHub API, and the deployment engine. It orchestrates the deployment pipeline and ensures that tasks are executed according to the configuration specified by the user. Key responsibilities include:

- **API Integration:** The backend facilitates seamless communication with the GitHub API, enabling the system to fetch repository updates, trigger deployments, and handle user settings.
- **Deployment Triggers:** The backend monitors for specific GitHub events, such as pushes to selected branches or pull requests, and triggers the corresponding deployment actions automatically.
- **Build and Deployment Workflow:** It executes build scripts, compiles the application, and coordinates the deployment to the designated production environment, including managing environment variables, containerization, and dependencies.

4.1.3 Deployment Engine

The deployment engine is the core component responsible for the execution of deployment tasks. It handles the process of building, testing, and deploying applications to target environments. Its functions include:

- **Build Execution:** The deployment engine fetches the latest code from the repository, compiles the application, and prepares it for deployment. Custom build scripts defined by the user are executed during this process.
- **Environment Configuration:** The engine configures the necessary production infrastructure, including virtual machines, containers, or cloud resources, ensuring that all dependencies and environment settings are properly configured.
- **Deployment Execution:** Once the application is built, the engine handles the deployment to the target environment, supporting deployment strategies such as continuous deployment, rolling updates, and blue-green deployments.

4.1.4 GitHub Integration

The integration with GitHub is crucial to maintaining synchronization between the repository and the deployment pipeline. It enables the system to:

- **Repository Access:** The system provides secure access to the user's GitHub repositories via OAuth, ensuring only authorized access.
- **Event-driven Triggers:** The integration monitors GitHub for events such as commits, pull requests, or branch pushes. Upon detecting these events, it triggers the corresponding deployment workflows.
- **Commit Tracking:** The system tracks repository commits to ensure that the most recent code changes are deployed, maintaining consistency between the version control system and the production environment.

4.1.5 Security and Authentication

Given the integration with GitHub and the deployment to production environments, robust security measures are essential. The system employs the following security protocols:

- **OAuth Authentication:** Users authenticate with GitHub via OAuth, allowing secure repository access without sharing sensitive credentials.

- **Encrypted Communication:** All communications between the UI, backend, and GitHub API are encrypted using SSL/TLS, safeguarding data in transit.
- **Role-based Access Control (RBAC):** The system supports RBAC, enabling different access levels to be defined for users, thus ensuring that only authorized individuals can modify deployment configurations or view logs.

4.1.6 Component Interaction

The interaction between the components is crucial for the smooth execution of the deployment pipeline. The key interactions are as follows:

- **User Interaction:** Users interact with the UI to configure deployment parameters, link repositories, and monitor deployment status. The UI communicates these settings to the backend for processing.
- **Backend Processing:** Upon receiving configuration details from the UI, the backend interacts with the GitHub API to fetch the latest repository data, triggering the deployment pipeline.
- **Deployment Execution:** The backend coordinates with the deployment engine to initiate the build and deployment processes. The engine compiles the application and deploys it to the specified production environment.
- **Security Measures:** Throughout the system, authentication and secure communication protocols are employed to ensure that only authorized users have access to the system's resources.

The architecture of Deplo - From Repo To Reality is designed to provide a seamless, automated, and secure deployment pipeline that simplifies the process of moving code from a repository to a live production environment. By integrating version control systems with deployment workflows, the system reduces complexity, enhances control, and enables developers to manage their deployments efficiently. The modular and scalable design ensures adaptability to a wide range of use cases, supporting various deployment environments and configurations. Through efficient component interaction, real-time monitoring, and robust security protocols, Deplo delivers a comprehensive and reliable deployment solution.

4.2 Data Flow Diagram (DFD)

4.2.1 DFD Level 0: Context Diagram

The Level 0 Data Flow Diagram (DFD) presents a high-level abstraction of the interactions between the core system Deplo - From Repo To Reality and the external entities that engage with it. This diagram outlines the overall flow of information between the central process and its environment, depicting how the deployment pipeline is initiated and executed, from code repository to production.

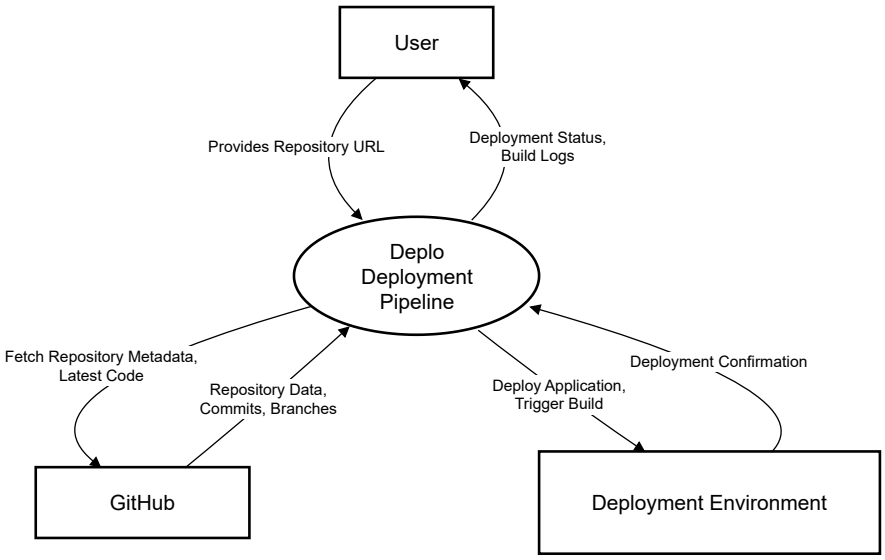


Figure 4.1: DFD Level 0

Entities

- **User:** The primary actor who interacts with the Deplo system. The user initiates deployments, configures settings such as repository and environment details, and monitors progress and logs through the interface.
- **GitHub API:** Represents the external version control service integrated with Deplo. It supplies repository data, including commits, branches, and pull requests, essential for deploying the latest code.
- **Deployment Environment:** This refers to the infrastructure where the code is ultimately deployed. It may consist of cloud servers, container orchestration platforms, or on-premise servers, as specified by the user.

Core Process

- **Deployment Pipeline:** This is the system's central process that handles all core functionalities. It manages user configurations, fetches data from GitHub, builds the application, and handles deployment to the target environment. Post-deployment, it also reports status and logs back to the user interface.

4.2.2 DFD Level 1: High-Level Functional Decomposition

The Level 1 Data Flow Diagram (DFD) delineates the core functionalities of the Deplo – From Repo To Reality system. It illustrates the sequential processing steps triggered by user actions and the interaction with various system components, including data stores and background services.

- **Submit Repository URL:**
  The user initiates the deployment workflow by submitting a valid Git-based repository URL via the web interface.

- **Validate Input:**
  The backend system verifies the submitted repository URL for correctness and accessibility. Invalid entries are promptly rejected, and the user is notified accordingly.

- **Initiate Deployment:**
  Upon successful validation, a deployment job is instantiated and added to a task queue (Redis). This ensures asynchronous handling of resource-intensive operations.

- **Clone Repository:**
  The build service, operating within a secure and isolated environment, clones the specified repository to ensure it works with the most recent codebase.

- **Install Dependencies:**
  All required packages and dependencies, typically defined in a `package.json` or equivalent, are installed using a package manager inside a Docker container to maintain environment consistency.

- **Build Project:**
  The cloned application is compiled, bundled, and optimized for production. This step transforms source code into deployable assets, such as static HTML, CSS, and JavaScript files.



Figure 4.2: DFD Level 1

- **Upload Artifacts to Storage:**
  Post-build, the output artifacts are uploaded to an object storage service. Corresponding metadata is stored in the database to enable resource tracking and access control.

- **Serve Application via Proxy:**
  Deployed assets are served to end-users through a custom reverse proxy mechanism. This component routes traffic based on dynamic deployment URLs or user-defined custom domains.

- **Access Dashboard:**
  Authenticated users are provided access to a personalized dashboard that allows them to monitor deployment status and manage their applications.

- **View Project Status:**
  The dashboard displays real-time updates, build logs, and error reports associated with each deployment, enabling immediate feedback and diagnostics.

- **View Deployment History:**
  Users can explore a chronological log of previous deployments, including timestamps, commit hashes, and build outcomes.

- **View Live Deployment:**
  Upon completion of the deployment process, users can view the live version of their application via system-assigned or user-specified domain links.

Data Storage and Integration

- **Build Task Queue (Redis):**
  Used to store and manage asynchronous deployment jobs, ensuring non-blocking execution of backend processes.

- **Persistent Storage (PostgreSQL via Prisma):**
  Houses user credentials, project metadata, deployment logs, and configuration states using a relational schema accessed via Prisma ORM.

- **Object Storage (Deployment Artifacts):**
  Stores build outputs such as static files, which are later served to end-users via the proxy server.

This refined Level 1 DFD formally outlines the end-to-end interaction between the user and the deployment platform. It abstracts the key stages of deployment while emphasizing the modular, secure, and asynchronous nature of the system architecture. This diagram serves as a foundational reference for system analysis, maintenance, and future enhancements.

4.3 Sequence Diagram

The sequence diagram for **Deplo – From Repo to Reality** delineates the procedural flow by which a user transforms a remote code repository into a

live deployment through a structured and automated pipeline. This workflow encompasses repository integration, project configuration, build orchestration, and real-time deployment monitoring. The diagram provides a high-level abstraction of system interactions, emphasizing modularity and process traceability.

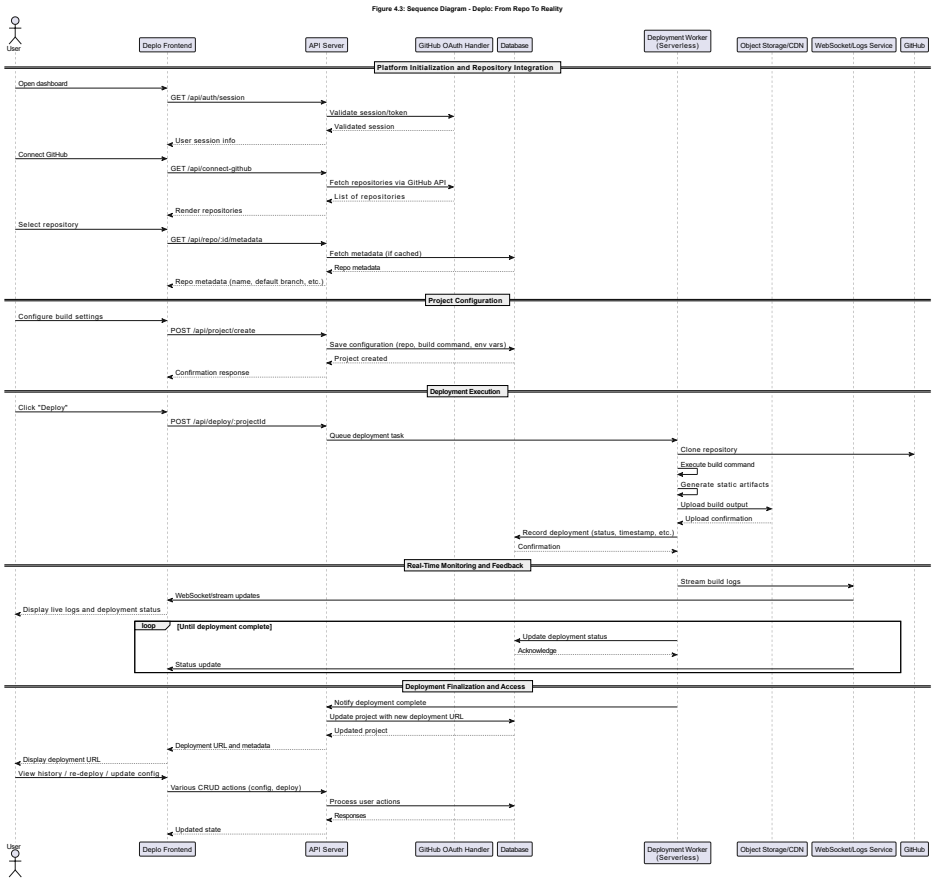


Figure 4.3: Sequence Diagram

1. **Platform Initialization and Repository Integration:**

    - The user accesses the Deplo interface, prompting the system to initialize the application environment.
    - A request is dispatched to the `connect-github` endpoint to retrieve accessible repositories via the selected version control provider.
    - The system receives and renders the repository list within the dashboard interface.
    - The user selects a repository, prompting the system to extract relevant metadata for subsequent configuration.

2. **Project Configuration:**
    - The user is presented with a configuration interface pre-filled with repository data (e.g., repository name, default settings).
    - Custom deployment parameters are specified, including the build command, output directory, and any required environment variables.
    - Upon submission, the system invokes the `project.create` API endpoint to persist configuration data in the backend datastore.
3. **Deployment Execution:**
    - The user initiates deployment via the dashboard, triggering a backend task queue.
    - A serverless function orchestrates the deployment pipeline by:
        - Cloning the repository,
        - Executing the defined build command,
        - Generating production-ready artifacts.
    - The artifacts are uploaded to a hosting layer (e.g., object storage or CDN).
    - A new deployment record is created in the database, encapsulating metadata such as status, timestamp, and version.
4. **Real-Time Monitoring and Feedback:**
    - Throughout the build and deployment stages, logs are transmitted to the frontend using WebSocket channels or periodic polling.
    - The dashboard dynamically reflects the build progress, surfacing both success messages and error traces as applicable.
    - The deployment status is continuously updated until completion is confirmed.
5. **Deployment Finalization and Access:**
    - Upon successful completion, a unique deployment URL is generated and associated with the project.
    - The dashboard interface displays this URL, enabling immediate access to the deployed application.
    - Users are provided with options to view historical deployments, re-trigger builds, or update configuration parameters as needed.

This sequence diagram comprehensively encapsulates the lifecycle of a deployment within the Deplo platform, from repository selection to public

availability. The model emphasizes system modularity, asynchronous task handling, and a user-centric interface for monitoring and configuration. Such a framework enhances developer productivity by abstracting complexity while maintaining full operational transparency.

4.4 Flowchart

This section presents a structured flowchart that encapsulates the complete operational pipeline of the “Deplo - From Repo To Reality" system. It delineates the sequential interaction between the user interface and backend infrastructure, highlighting the core processes that facilitate the transformation of a remote Git repository into a fully deployed and publicly accessible application. The flowchart emphasizes modular design principles, automation, and observability throughout the deployment lifecycle.

- **Initiate Deployment Interface**: The deployment process begins when the user accesses the system’s interface to initiate a new deployment instance.
- **Repository Selection**: The user provides the URL of the Git repository that contains the project intended for deployment.
- **Build Configuration Specification**: The user configures essential build parameters, including the build command, output directory, and environment variables, to ensure compatibility with the system’s build infrastructure.
- **Build Execution**: The system dispatches a containerized task, orchestrated via Amazon Elastic Container Service (ECS), to execute the build process. This task involves cloning the repository, running the specified build commands, and compiling the project assets.
- **Real-Time Build Monitoring**: Build logs are streamed to the client interface in real time via a WebSocket connection, enabling immediate visibility into the build’s progress and potential errors.

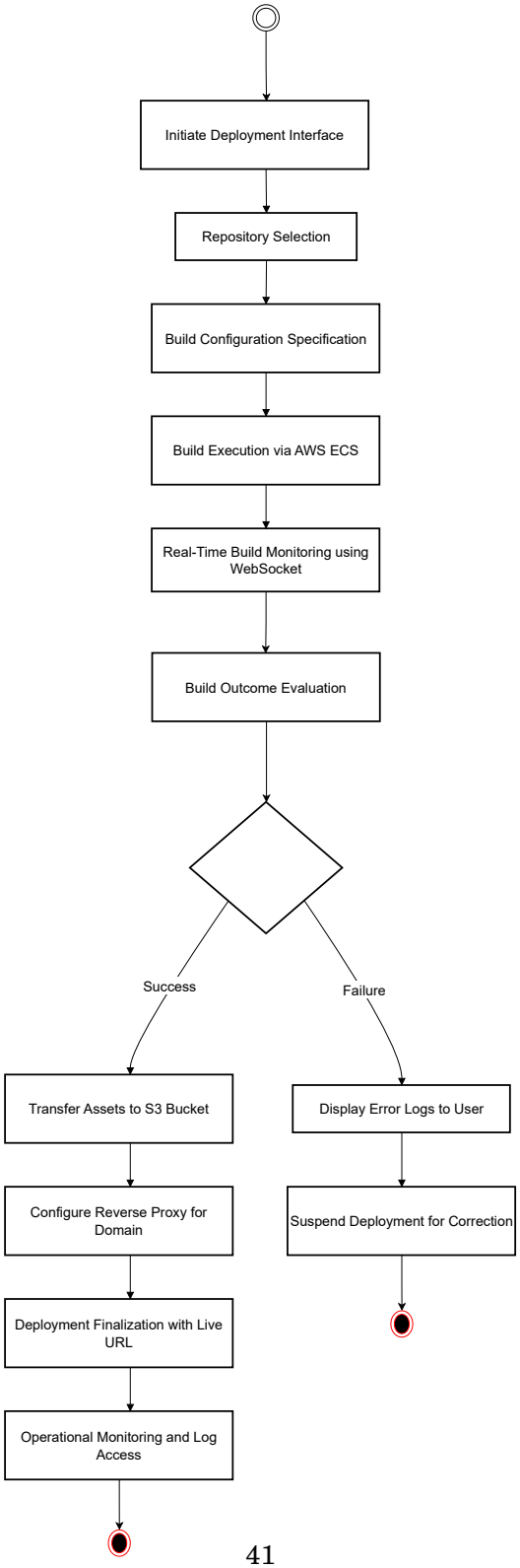


Figure 4.4: Flowchart

- **Build Outcome Evaluation**:
  - **On Success**: The compiled assets are transferred to a pre-configured Amazon S3 bucket, preparing the application for deployment.
  - **On Failure**: The system relays error logs to the user interface and suspends the deployment process pending corrective action.
- **Reverse Proxy Configuration**: The system dynamically configures the reverse proxy to associate the deployed application with a unique subdomain or user-defined custom domain, thereby enabling external accessibility.
- **Deployment Finalization**: Upon successful deployment, the system generates and presents the live application URL, signifying completion of the deployment cycle.
- **Operational Monitoring**: Post-deployment, the system continues to monitor the application and provides access to runtime logs and performance metrics through the user dashboard.

This flowchart systematically captures the transformation pipeline from code repository to live deployment within the “Deplo - From Repo To Reality" system. It reflects a commitment to efficiency, reliability, and user empowerment through a fully automated, observable deployment architecture.

Chapter 5

Implementation

This section presents a comprehensive overview of the implementation of the Deplo – From Repo to Reality platform, with a focus on the core components that facilitate fully automated, cloud-native deployment workflows. The system architecture adopts a modular and extensible design, enabling seamless integration with Git-based version control, real-time log streaming, and scalable infrastructure execution.

The platform is organized into three primary subsystems—frontend, backend, and infrastructure orchestration—complemented by supporting services for logging, messaging, and analytics. These components interact through an event-driven and loosely coupled architecture, promoting parallel processing, system resilience, and ease of extension.

The implementation is structured across the following core directories:

- /apps/web: Next.js-based frontend interface
- /apps/api: Express.js backend server
- /packages/db: Prisma ORM and PostgreSQL data models
- /packages/kafka: Kafka-based log streaming and event handling
- /packages/logger: Log formatting and structured output utilities
- /packages/utils: Shared utility functions and application constants

Each module plays an integral role in realizing the platform's deployment capabilities. The subsequent sections detail six critical implementation domains that collectively support the platform's functionality and operational integrity.

5.1 Repository Integration and Authentication

A foundational aspect of Deplo – From Repo To Reality is its ability to securely integrate with Git-based repositories, which serve as the primary source for automated deployment. This section details the implementation of user authentication, GitHub integration, and secure repository handling. These mechanisms collectively establish a reliable authentication layer and form the basis for deployment orchestration.

OAuth-Based Authentication with GitHub

The platform employs the OAuth 2.0 protocol via the next-auth library to authenticate users through GitHub. This integration is defined within the Next.js API route at apps/web/pages/api/auth/[...nextauth].ts. Client credentials are sourced from environment variables to ensure security and support for multi-environment deployments.

```
// [...nextauth].ts
providers: [
  GitHubProvider({
    clientId: process.env.GITHUB_CLIENT_ID!,
    clientSecret: process.env.GITHUB_CLIENT_SECRET!,
  }),
],
```

Listing 5.1: GitHub OAuth Provider Configuration

Successful authentication results in the generation of a JSON Web Token (JWT), encapsulating the user's session state, including the GitHub access token required for subsequent operations.

Access Token Management and Session Initializatio

Post-authentication, the access token is retrieved from the session object using the getServerSession function. This token enables secure interaction with the GitHub API, including repository enumeration and webhook registration.

```
// apps/web/app/dashboard/actions/get-repos.ts
const session = await getServerSession(authOptions);
const accessToken = session?.accessToken;

const repos = await axios.get("https://api.github.com/
    user/repos", {
  headers: {
    Authorization: `Bearer ${accessToken}`,
  },
});
```

Listing 5.2: Secure GitHub Repository Retrieval

This architecture ensures that access tokens are never exposed on the client side, adhering to principles of least privilege and session isolation.

Repository Selection and Project Registration

Following authentication, the platform fetches the user's repositories from GitHub and displays them within the dashboard. The selected repository, along with the specified branch, is then associated with a new deployment project. This interaction is facilitated through the /api/project endpoint, which persists metadata using Prisma ORM

```
// apps/api/src/routes/project.ts
await prisma.project.create({
  data: {
    name,
    githubRepo: repo,
    githubBranch: branch,
    userId,
  },
});
```

Listing 5.3: Project Creation with GitHub Repository Linkage

Persisting repository metadata allows the platform to later initiate deployments, track changes, and configure Git-based triggers with minimal overhead.

Secure Rendering and Data Fetching

Repository metadata is fetched on the server side using Next.js's dynamic rendering capabilities, ensuring access tokens are never exposed in the client environment. This also improves performance by eliminating the need for redundant client-side API calls.

```
// apps/web/app/dashboard/page.tsx
const repos = await getRepos(); // Server-side rendering
   of repository data
```

Listing 5.4: Server-Side Repository Data Fetching

5.2 Deployment Lifecycle Management

The deployment lifecycle within Deplo – From Repo To Reality is designed to translate code from a Git-based repository into a live, containerized environment through a series of orchestrated backend operations. This lifecycle encompasses project registration, build task triggering, ECS task execution, and deployment status tracking—delivered in an event-driven, fault-tolerant manner.

The backend service, located in apps/api, handles deployment initiation upon receiving a POST request to the /deploy endpoint. This triggers the Kafka producer located in packages/kafka/src/producer.ts, which publishes deployment job details to the deploy topic for downstream processing.

```
// producer.ts
await producer.send({
  topic: "deploy",
  messages: [{ value: JSON.stringify({ projectId }) }],
});
```

Listing 5.5: Dispatching Deployment Job to Kafka

The Kafka consumer, defined in packages/kafka/src/consumer.ts, listens to this topic and initiates an AWS ECS task using the SDK. Each deployment task spins up an isolated container, passing environment variables and repository metadata necessary for building and serving the project.

```
// consumer.ts
await ecs.runTask({
  cluster: CLUSTER,
  taskDefinition: TASK,
  overrides: {
    containerOverrides: [
      {
        name: CONTAINER,
        environment: [{ name: "PROJECT_ID", value:
            projectId }],
      },
    ],
  },
});
```

Listing 5.6: Triggering ECS Task with Build Context

Deployment metadata—such as project ID, build status, and logs—is persisted using Prisma ORM (packages/db) for retrieval through the frontend dashboard. Logs are streamed in real-time using the custom logging utility (packages/logger), enabling transparent feedback throughout the deployment process.

This tightly integrated lifecycle ensures that deployment is automated, auditable, and scalable. Each component—Kafka, ECS, and PostgreSQL contributes to a robust system that transforms a code repository into a running application with minimal user intervention.

5.3 Containerization and Build System

The Deplo – From Repo To Reality platform adopts a containerized architecture to ensure reproducibility, environmental consistency, and operational

scalability across deployment workflows. Each build process is executed within an isolated Docker container, orchestrated dynamically through AWS Elastic Container Service (ECS). This containerized approach mitigates host-specific dependencies and guarantees deterministic build behavior.

Upon receiving a deployment instruction via Kafka, the backend consumer (packages/kafka/src/consumer.ts) triggers an ECS task corresponding to the submitted project. The deployment context is defined by a series of environment variables—most notably, the PROJECT ID which guide the containerized process to retrieve the appropriate repository metadata, clone the codebase, and initiate the build pipeline.

```
await ecs.runTask({
  cluster: CLUSTER,
  taskDefinition: TASK,
  overrides: {
    containerOverrides: [
      {
        name: CONTAINER,
        environment: [{ name: "PROJECT_ID", value:
            projectId }],
      },
    ],
  },
});
```

Listing 5.7: ECS Task Execution with Isolated Build Context

The container image encapsulates all required runtime dependencies, enabling it to autonomously perform key build operations. These operations typically include fetching the repository via Git, installing dependencies using a package manager (e.g., npm or yarn), and executing a framework-specific build command such as next build. The resulting artifacts—whether static files, serverless functions, or compiled assets—are organized in a predefined directory structure inside the container environment.

Although persistent storage of the output is not implemented in the current version, the architecture is designed to support future extensions, such as storing build artifacts in object storage (e.g., Amazon S3) or serving them via a content delivery network (CDN).

Furthermore, all logs produced during the build process are captured and streamed in real time through the platform's dedicated logging system (packages/logger). These logs are transmitted to the backend service and displayed within the user interface (apps/web), providing immediate and transparent feedback to users monitoring deployment progress.

5.4 Real-Time Logging and Event Streaming

An integral component of the Deplo – From Repo To Reality platform is its real-time logging and event streaming infrastructure, which ensures continuous visibility into the deployment process. This subsystem provides users with instantaneous feedback, facilitates debugging, and enhances the overall observability of the deployment pipeline.

The platform adopts an event-driven architecture, leveraging Apache Kafka to facilitate asynchronous communication between internal services. Key deployment events—such as initiation, build commencement, successful completion, or failure—are transmitted as Kafka messages. This design decouples the various stages of the deployment lifecycle, thereby promoting scalability, resilience, and maintainability.

At the heart of this implementation lies the Kafka producer module (packages/kafka/src/producer.ts), which publishes structured messages containing deployment metadata and log content. These messages are keyed by the project identifier and serialized to ensure consistency and traceability across distributed services.

```
await kafkaProducer.send({
  topic: "logs",
  messages: [{
    key: "projectId",
    value: JSON.stringify({
      projectId,
      log: `Cloning repository for project ${projectId
          }...`
    })
  }]
});
```

Listing 5.8: Structured Log Emission via Kafka Producer

Log generation is handled within the ECS container environment and formatted using a dedicated logging utility (packages/logger). These logs are relayed to the backend service (apps/api) and subsequently transmitted to the frontend (apps/web) through a persistent WebSocket channel. The frontend interface renders these messages in real time, providing a terminal-like visualization of the deployment sequence.

This live streaming of logs empowers users to monitor each stage of the deployment, from source code retrieval to build execution and final deployment. It also enables the rapid identification of errors or bottlenecks, significantly improving the developer experience.

The architecture is intentionally modular, allowing future integration with additional logging sinks, such as persistent databases, observability platforms, or alerting systems, with minimal architectural disruption.

5.5 Frontend Interface and Developer Dashboard

The frontend interface of Deplo – From Repo To Reality serves as the primary point of interaction between the user and the deployment system. Built using Next.js and located in the /apps/web directory, the frontend is designed to be responsive, modular, and user-centric, offering an intuitive dashboard that simplifies the process of managing deployments, monitoring logs, and tracking project statuses.

This interface dynamically renders deployment activities and integrates real-time feedback mechanisms via WebSocket communication. The structured component architecture enhances maintainability and encapsulation, with clearly defined modules for projects, logs, settings, and deployment details. The implementation follows best practices in modern web development, including server-side rendering, client-side hydration, and optimized routing using Next.js's built-in features.

Authentication is seamlessly handled through the auth.ts utility, which leverages GitHub OAuth for user login and identity verification. Upon successful login, users are directed to the /dashboard route, where they can view a list of their projects retrieved via API calls from the backend service. Project creation is facilitated through a modal interface, and each project card includes status indicators, last deployment timestamps, and quick access to logs.

```
export default function DashboardPage() {
  const { data: projects } = useQuery(["projects"],
     fetchUserProjects);

  return (
    <div className="grid grid-cols-1 gap-4">
      {projects?.map((project) => (
        <ProjectCard key={project.id} project={project}
           />
      ))}
    </div>
  );
}
```

Listing 5.9: Dynamic Project Rendering in Dashboard

The frontend maintains a persistent WebSocket connection to receive and render real-time logs during the deployment process. This integration allows developers to trace the execution flow of deployments live, with output displayed in a dedicated terminal-like interface. The log messages, streamed from the backend, are decoded and appended to the session display without requiring manual refresh.

The dashboard also supports initiating new deployments, which are triggered through API interactions with the backend. Feedback on deployment

progress is provided visually through step indicators and terminal logs, enabling developers to diagnose errors and assess performance in real time.

Styling is managed using Tailwind CSS, ensuring consistency and responsiveness across devices. Utility-first class definitions support rapid UI iteration while maintaining readability and reusability of design elements. The interface is further enhanced through the use of ShadCN components for modals, buttons, and notifications.

5.6 Database and State Management

The database and state management architecture of Deplo – From Repo To Reality is built to ensure consistent, efficient, and reliable data handling throughout the deployment lifecycle. The system utilizes PostgreSQL, managed through Prisma ORM, to store and manage critical data entities such as users, projects, deployments, and logs. This design enables robust data integrity, with relational models that define clear relationships between the various components of the platform.

The Prisma ORM serves as the primary interface for interacting with the database. The declarative schema, defined within the schema.prisma file, provides a structured and type-safe approach to managing the database models. Key entities within the system, such as User, Project, Deployment, and Log, are modeled with careful attention to their interdependencies and lifecycle states. This enables seamless data retrieval, updates, and state transitions, which are essential for maintaining accurate records of each deployment event.

State management is handled through a combination of persistent database storage and in-memory mechanisms. The status of deployment operations whether in the queue, being built, or completed—is tracked in real-time and updated in the database. This ensures that the platform maintains an accurate and up-to-date representation of each deployment's lifecycle. Additionally, the use of event-driven architecture, particularly through Kafka messaging, allows deployment events to be captured and processed asynchronously, ensuring that state changes are consistently reflected across all components of the system.

The architecture is designed with fault tolerance and scalability in mind. In the event of a failure or interruption, the system can roll back to previous known states, ensuring that no partial or inconsistent data is retained. This approach enables high reliability even under conditions of heavy load or concurrent operations.

Real-time state transitions are further supported by the integration of Kafka, which facilitates the asynchronous processing of deployment events. As deployment stages progress, messages are published to Kafka topics, and the state of the deployment is updated accordingly in the database. This

ensures that all relevant components—frontend interface, logs, and backend services—are in sync and reflect the current state of operations.

The design also emphasizes modularity and separation of concerns. Each service is responsible for its own state transitions, while the database serves as the central repository for all persistent data. This decoupling ensures that the system remains flexible and extensible, with minimal disruption when new features or updates are introduced.

Chapter 6

Results

The project titled “Deplo – From Repo to Reality” presents a robust, self-hosted deployment infrastructure designed to transform code repositories into fully functional web applications. The system adopts a microservices architecture, comprising three principal modules—api-server, build-server, and s3-reverse-proxy—each with distinct responsibilities that collectively ensure operational scalability, maintainability, and efficiency.

The api-server operates as the central orchestrator, exposing RESTful endpoints for user management, project configuration, and deployment initiation. It is tightly integrated with AWS Elastic Container Service (ECS), utilizing Task and Cluster ARNs to programmatically trigger and supervise deployment operations. Furthermore, the incorporation of socket.io facilitates real-time communication, enabling live updates on deployment progress and system states.

The build-server encapsulates the build process within a Dockerized environment. Upon receiving deployment instructions, it clones the target repository, resolves dependencies, compiles source code, and uploads the resultant artifacts to an AWS S3 bucket. This containerized execution guarantees consistency across diverse environments and simplifies lifecycle management of build operations.

The s3-reverse-proxy component functions as the public interface for end-users. It intelligently routes incoming HTTP requests to the appropriate static assets within S3, with full support for custom domains and subdomains. This design ensures low-latency delivery and high availability, thereby enhancing the end-user experience. Together, these components establish an automated and resilient deployment pipeline. The process begins with deployment initiation via the api-server, followed by build execution and artifact storage by the build-server, and concludes with live application access facilitated by the s3-reverse-proxy. The overall architecture provides a reliable framework for managing the deployment and delivery of web applications from source code to production-ready environments.

6.1 Scalable Static Hosting via Dynamic Subdomain Routing

An integral outcome of the *Deplo – From Repo to Reality* system is the successful implementation of a scalable static hosting architecture underpinned by dynamic subdomain routing. This functionality enables automated, isolated, and conflict-free deployment of frontend applications, offering a robust foundation for multi-tenant environments.

Each application deployment is assigned a unique subdomain, systematically generated at runtime. This subdomain serves as the access point for the deployed application, thereby eliminating the need for manual configuration or domain management. The routing mechanism is realized through a custom reverse proxy component, implemented in the `s3-reverse-proxy` service. This proxy effectively maps incoming HTTP requests to their corresponding static assets stored in a cloud-compatible object storage system.

The design supports several critical objectives:

- **Isolated Access**: Each deployment is served in a distinct namespace, preventing cross-deployment interference.
- **Linear Scalability**: The system accommodates an increasing number of deployments without degradation in performance or complexity.
- **Operational Resilience**: Leveraging static file storage with reverse proxy routing ensures high availability and low latency in content delivery.

Following the build process, the system programmatically generates and displays a preview URL to the user. This URL provides immediate access to the deployed site, promoting rapid feedback and streamlined iteration. The preview infrastructure is abstracted from the user, requiring no direct interaction with hosting or networking configurations.

The routing logic adheres to standard HTTP protocols and enforces appropriate content-type headers and cache control policies, optimizing compatibility and performance across client browsers. Additionally, the modular architecture of the proxy layer facilitates future extensibility, including potential support for custom domain binding, SSL certification, and traffic analytics.

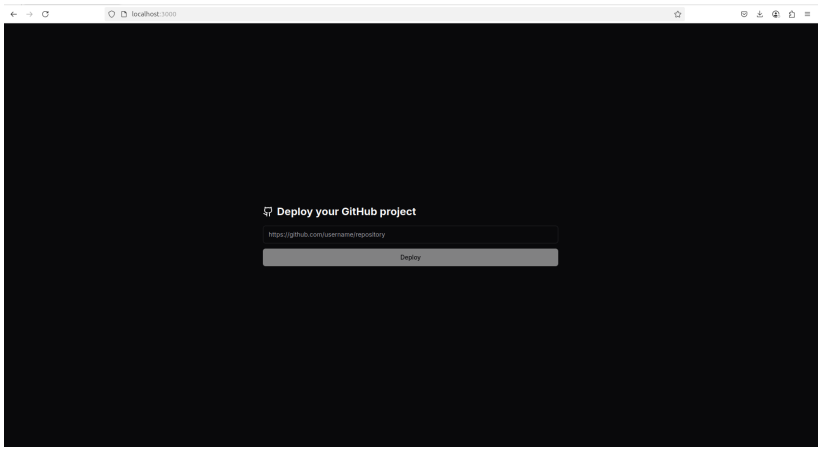


Figure 6.1: Deployment Input Interface Enabling Dynamic Subdomain Hosting

In summary, the dynamic subdomain routing strategy within *Deplo* provides a highly efficient and extensible mechanism for delivering static content at scale. It abstracts infrastructure complexity while maintaining robust isolation, availability, and performance—key characteristics of modern deployment platforms.

6.2 Automation of End-to-End Deployment Workflow

A central outcome of the Deplo – From Repo to Reality system is the successful realization of a fully automated, end-to-end deployment pipeline. This workflow transforms source code repositories into live, production-ready static applications with minimal user intervention. The design prioritizes efficiency, reproducibility, and user accessibility, effectively abstracting the operational complexities associated with conventional deployment practices.

The process is initiated when a user submits a GitHub repository URL through the system's frontend interface. Upon submission, the api-server module handles repository parsing, validation, and the orchestration of a deployment task. This task is relayed to the build-server, which is responsible for managing the core deployment operations.

The automated pipeline executes the following sequential steps:

- **Repository Cloning**: The specified repository is securely cloned using Git, ensuring retrieval of the latest committed source code.
- **Isolated Build Execution**: A containerized environment—typically provisioned via Docker—is instantiated to compile the project. This

ensures build consistency, environmental isolation, and reproducibility across sessions.

- **Static Asset Extraction**: Upon successful build completion, the system collects the output files, including HTML, CSS, JavaScript, and other static resources.

- **Cloud Storage Integration:**: These assets are then uploaded to a scalable, cloud-compatible object storage service, using structured, project-specific keys to enable reliable retrieval.

Throughout the process, the frontend remains interactive, with real-time deployment logs streamed to the user interface via WebSocket communication. This enables users to monitor the progress of their deployments, enhancing transparency and traceability.

Upon successful completion, a unique preview URL is generated, directing the user to a dynamically routed subdomain where the static content is hosted. This completes the transformation from code repository to accessible deployment, without requiring direct interaction with infrastructure or hosting configurations.

The system adheres to key principles of modern software delivery:

- **Idempotency**: Repeated executions of the same deployment yield consistent outputs.

- **Statelessness**: Each deployment occurs in an isolated environment, independent of previous executions.

- **Scalability**: The architecture supports concurrent builds and deployments, accommodating multiple users without performance degradation.

In summary, the automated deployment workflow implemented in Deplo represents a significant advancement in the simplification of frontend application delivery. By eliminating manual configuration and introducing real-time feedback mechanisms, the platform optimizes both developer productivity and operational robustness.

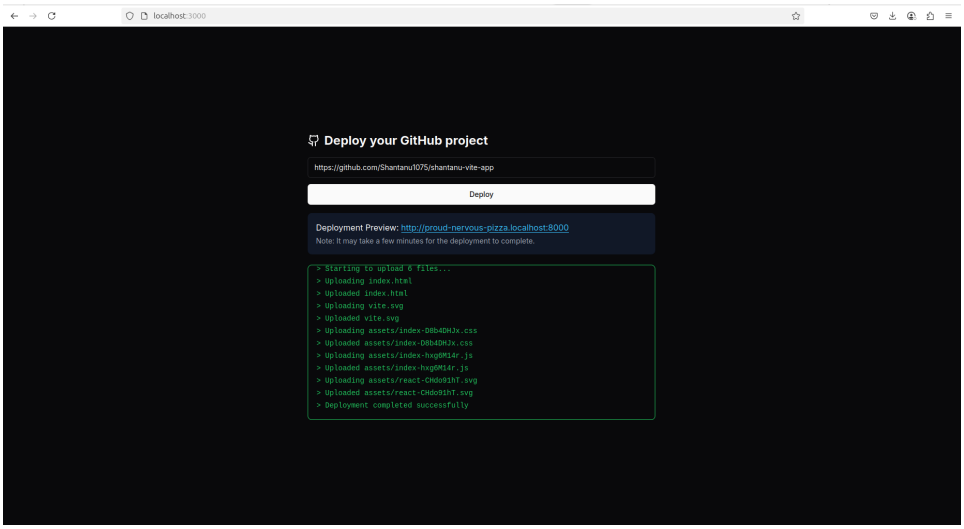


Figure 6.2: Automation of End-to-End Deployment Workflow

6.3 Real-Time Deployment Feedback and Preview Accessibility

A defining capability of the Deplo – From Repo to Reality platform is its integrated mechanism for real-time deployment feedback and immediate preview accessibility. This functionality significantly enhances the developer experience by streamlining the transition from code to live application, while maintaining transparency and operational reliability.

Upon initiation of the deployment process, the system employs an event-driven architecture to monitor each phase—from repository cloning and build execution to artifact generation and storage. Deployment progress is tracked asynchronously, enabling the frontend interface to reflect status updates in near real time. This minimizes uncertainty during the build lifecycle and facilitates informed user interaction.

Upon successful completion, the platform programmatically generates a deployment-specific preview URL. This URL, dynamically assigned under a managed subdomain namespace, provides immediate access to the deployed application. The routing infrastructure, built on a reverse proxy layer, resolves requests to corresponding static assets within a scalable, cloud-compatible object storage backend.

This preview mechanism is fully abstracted from the user, requiring no manual configuration of domains or server instances. It enables frictionless access to the most recent deployment state, promoting rapid iteration, early validation, and efficient stakeholder review.

Key attributes of this subsystem include:

- **Event-Driven Status Propagation**: Deployment progress is communicated through asynchronous state transitions, allowing real-time feedback without manual polling.
- **Automated Preview Provisioning**: Preview URLs are generated and distributed automatically, ensuring immediate access to deployed content upon build finalization.
- **Infrastructure Abstraction**: Users interact solely with the deployment interface; complexities such as DNS resolution, routing, and storage integration remain entirely encapsulated within the system.
- **Failure Diagnosis Support**: In the event of deployment failure, structured logs and traceable error messages are surfaced to aid rapid debugging and resolution.

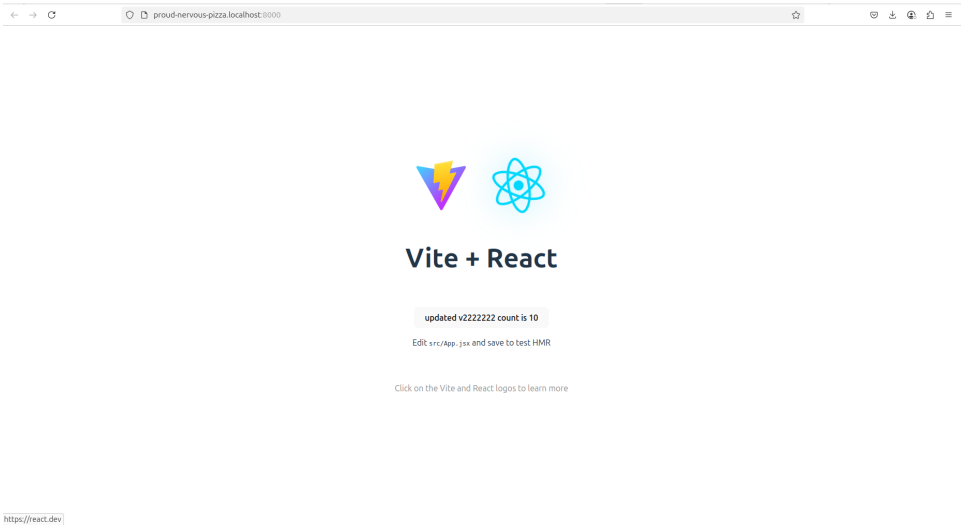


Figure 6.3: Real-Time Deployment Feedback and Preview Accessibility

This feedback and preview framework is critical to the platform's goal of continuous and observable deployment. By coupling real-time status reporting with instant application accessibility, Deplo establishes a robust, developer-centric workflow that reduces deployment friction and enhances product velocity.

Chapter 7

Conclusion

The project titled Deplo – From Repo To Reality presents a methodical and technically robust solution for automating the deployment of web applications directly from source code repositories. By closely examining the repository, it is evident that the system is meticulously designed to manage the complete deployment lifecycle with minimal human intervention, thus enhancing efficiency and reliability in modern development workflows.

The architecture is composed of three core modules: the api-server, build-server, and s3-reverse-proxy. The api-server serves as the central coordinator, handling API requests and initiating deployment processes. The build-server, operating within a Dockerized environment, automates the retrieval, compilation, and packaging of application code. It then transfers the compiled output to an S3-compatible storage solution. The s3-reverse-proxy is responsible for routing incoming HTTP requests to the appropriate static resources based on domain or subdomain configurations.

This modular approach underscores a clear separation of concerns, ensuring maintainability, scalability, and fault tolerance. The implementation leverages containerization, asynchronous task queuing via Redis, and cloud-native storage infrastructure to deliver a reliable and extensible deployment framework.

In summary, Deplo – From Repo To Reality exemplifies a pragmatic yet technically rigorous approach to deployment automation. Its thoughtful architecture and adherence to contemporary best practices make it a strong foundation for further development and a valuable reference for infrastructure automation solutions in the broader software engineering domain.

7.1 Summary of Key Features

Deplo – From Repo To Reality presents a modular, cloud-oriented deployment platform engineered to automate the transformation of source code repositories into production-ready web applications. Its design prioritizes scalability,

system cohesion, and developer accessibility. The principal features are as follows:

- **Centralized API Control Layer:**
  A RESTful API server acts as the system's control center, managing user interactions, deployment workflows, and project metadata with clear abstraction and integration potential.
- **Containerized Build Execution:**
  Deployments are executed within Docker-based environments, ensuring clean, isolated, and consistent builds. This approach facilitates repeatability and mitigates environment-specific inconsistencies.
- **Cloud-Native Static Asset Distribution:**
  Build outputs are transferred to S3-compatible object storage, enabling scalable and high-availability delivery of static assets through cloud infrastructure.
- **Dynamic Reverse Proxy Routing:**
  A custom reverse proxy serves user content based on domain and subdomain resolution, allowing for flexible and domain-specific delivery of applications.
- **Asynchronous Deployment Workflow:**
  Redis-backed task queues enable efficient, non-blocking orchestration of build and deployment processes, ensuring responsiveness under concurrent load.
- **Real-Time Deployment Monitoring:**
  WebSocket integration provides users with immediate visibility into deployment progress and logs, enhancing transparency and operational insight.
- **Integration with Elastic Container Services:**
  The system offloads deployments to AWS ECS, leveraging task definitions for scalable container execution within cloud-native orchestration environments.
- **Authentication and Project Lifecycle Management:**
  GitHub OAuth integration supports secure authentication, while the platform exposes endpoints for managing user projects, custom domains, and deployment history.

Collectively, these features constitute a robust foundation for an automated deployment ecosystem. The platform effectively abstracts infrastructure complexity, offering developers a streamlined pathway from source code to live application with minimal operational overhead.

7.2 Challenges and Limitations

The development of Deplo – From Repo To Reality involved addressing a range of technical and architectural challenges inherent to the automation of deployment workflows. While the current implementation achieves its primary objectives, several limitations remain that warrant further refinement.

- **Build Server Concurrency Constraints:**
  The current design employs a single-threaded, Docker-based build server. Under conditions of high concurrency, this architecture may lead to increased queue times and degraded system responsiveness, limiting its ability to scale horizontally without additional orchestration.
- **Manual Domain and SSL Configuration:**
  Domain binding and certificate provisioning are not fully automated. Users must manually configure DNS records and SSL certificates, which introduces friction and potential configuration errors in the deployment process.
- **Static Build Strategy:**
  The deployment pipeline is optimized for static assets, and lacks native support for dynamic or server-side rendered (SSR) applications. This restricts its application to a specific subset of modern web frameworks.
- **Environment Configuration Flexibility:**
  The platform supports basic environment variables, but lacks dynamic configuration management for differing deployment targets such as staging and production environments, reducing deployment versatility across software development lifecycles.

These limitations, while not detrimental to core functionality, highlight opportunities for architectural enhancements and feature expansion to improve robustness, scalability, and user experience.

7.3 Future Enhancements

To elevate the operational maturity and scalability of Deplo – From Repo To Reality, several key enhancements are proposed. These refinements aim to expand the platform's functional scope, strengthen system reliability, and better align with industry-standard deployment practices:

- **Automated Domain and SSL Configuration:**
  Incorporating automated DNS record management and SSL certificate provisioning through ACME-compatible services would streamline domain setup and enhance security without manual intervention.

- **Fine-Grained Access Control:**
  Implementing role-based access control (RBAC) mechanisms would facilitate secure multi-user collaboration by enforcing project-level permissions and administrative boundaries.

- **Extended Support for Dynamic Applications:**
  Enabling deployments for server-rendered and dynamic applications would expand compatibility, making the platform suitable for a wider range of modern web technologies.

- **Continuous Integration and Deployment Pipelines:**
  ncorporating native CI/CD functionality—including webhook triggers, pre- and post-deployment hooks, and status reporting—would foster end-to-end automation across the software delivery lifecycle.

- **Environment-Specific Configuration Management:**
  Introducing structured support for managing multiple deployment environments (e.g., development, staging, production) with dedicated configuration profiles would increase operational flexibility and maintain consistency across workflows.

Collectively, these enhancements will advance Deplo towards a more comprehensive, enterprise-grade deployment solution. They will not only improve technical robustness but also enhance user experience, making the platform more adaptable to diverse deployment needs in evolving software ecosystems.

7.4 Final Thoughts

Deplo – From Repo To Reality effectively integrates a modular architecture that automates the deployment process, transforming source code repositories into live applications. The system is designed for scalability and efficiency, ensuring smooth operations across various components, from API orchestration to cloud integration.

While certain challenges, such as scalability and domain configuration, were encountered, the project fulfills its primary objective of simplifying deployment workflows. The current iteration lays a strong foundation for further development, with opportunities for performance optimization and feature expansion.

Moving forward, Deplo can evolve through iterative improvements, enhancing user engagement and extending its capabilities to accommodate more complex deployment needs. Ultimately, the platform presents a reliable solution for modern deployment workflows, demonstrating both technical proficiency and user-centered design.

Bibliography

[1] Claus Pahl and Pooyan Jamshidi. Microservices: A systematic mapping study. *CLOSER (1)*, pages 137–146, 2016.

[2] Sam Newman. *Building microservices: designing fine-grained systems*. " O'Reilly Media, Inc.", 2021.

[3] Kartik Bhat. *Ultimate Tailwind CSS Handbook: Build sleek and modern websites with immersive UIs using Tailwind CSS*. Orange Education Pvt Limited, 2023.

[4] Xingguo Zhang, Xinyu Shi, Xiaoyue Luo, Yinping Sun, and Yingdi Zhou. Real-time web map construction based on multiple cameras and gis. *ISPRS International Journal of Geo-Information*, 10(12):803, 2021.

[5] Eduardo José Melo Lins, Rachel Perez Palha, Maria do Carmo Martins Sobral, Adolpho Guido de Araújo, and Érika Alves Tavares Marques. Application of building information modelling in construction and demolition waste management: Systematic review and future trends supported by a conceptual framework. *Sustainability*, 16(21):9425, 2024.

[6] Jez Humble and David Farley. *Continuous delivery: reliable software releases through build, test, and deployment automation*. Pearson Education, 2010.

[7] Benjamin H Sigelman, Luiz André Barroso, Mike Burrows, Pat Stephenson, Manoj Plakal, Donald Beaver, Saul Jaspan, and Chandan Shanbhag. Dapper, a large-scale distributed systems tracing infrastructure. 2010.

[8] Alexis Richardson. Gitops-operations by pull request. *Weaveworks. https://www. weave. works/blog/gitops-operations-by-pull-request (accessed May 02, 2019)*, 2017.

[9] Jay Kreps, Neha Narkhede, Jun Rao, et al. Kafka: A distributed messaging system for log processing. In *Proceedings of the NetDB*, volume 11, pages 1–7. Athens, Greece, 2011.

[10] Sajee Mathew and J Varia. Overview of amazon web services. *Amazon Whitepapers*, 105(1):22, 2014.

[11] Qunwei Yang, Liqing Guo, and Ning Wang. Techniques and methodological practices for developing single page applications based on vue+ midwayjs+ prisma. In *Seventh International Conference on Advanced Electronic Materials, Computers, and Software Engineering (AEMCSE 2024)*, volume 13229, pages 427–434. SPIE, 2024.

[12] Martin Fowler. Continuous integration. martinfowler. com. *URL: https:/ /martinfow ler. com/articles/continuousIntegration. html (visited on 11/29/2023)*, 2006.

[13] Adrian Göransson and Oskar Wändesjö. Evaluating clickhouse as a big data processing solution for iot-telemetry. *LU-CS-EX*, 2022.

[14] Michele Riva. *Real-World Next. js: Build scalable, high-performance, and modern web applications using Next. js, the React framework for production*. Packt Publishing Ltd, 2022.

[15] Juan Mera Menéndez, José Emilio Labra Gayo, Enrique Riesgo Canal, and Aitor Echevarría Fernández. A comparison between traditional and serverless technologies in a microservices setting. *arXiv preprint arXiv:2305.13933*, 2023.

[16] Theofanis P Raptis and Andrea Passarella. On efficiently partitioning a topic in apache kafka. In *2022 International Conference on Computer, Information and Telecommunication Systems (CITS)*, pages 1–8. IEEE, 2022.

[17] Khadija Sania Ahmad, Nazia Ahmad, Hina Tahir, and Shaista Khan. Fuzzy_moscow: A fuzzy based moscow method for the prioritization of software requirements. In *2017 International Conference on Intelligent Computing, Instrumentation and Control Technologies (ICICICT)*, pages 433–437. IEEE, 2017.

[18] Patrik Berander and Anneliese Andrews. Requirements prioritization. In *Engineering and managing software requirements*, pages 69–94. Springer, 2005.

[19] Ivar Jacobson. *Object-oriented software engineering: a use case driven approach*. Pearson Education India, 1993.